

第5章 回溯法

1. 装载问题与01背包
2. 回溯算法设计步骤
3. 旅行售货员问题(TSP)
4. **n皇后问题**
5. 最大团问题
6. 符号三角形
7. 回溯算法的效率

n后问题

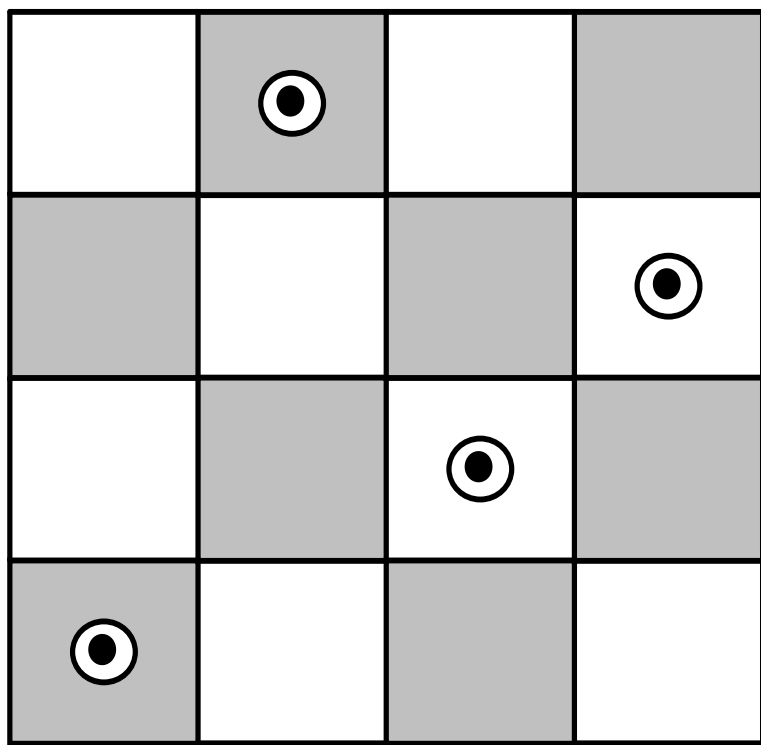
- 1.问题描述
- 2.问题分析
- 3.算法描述

1.问题描述

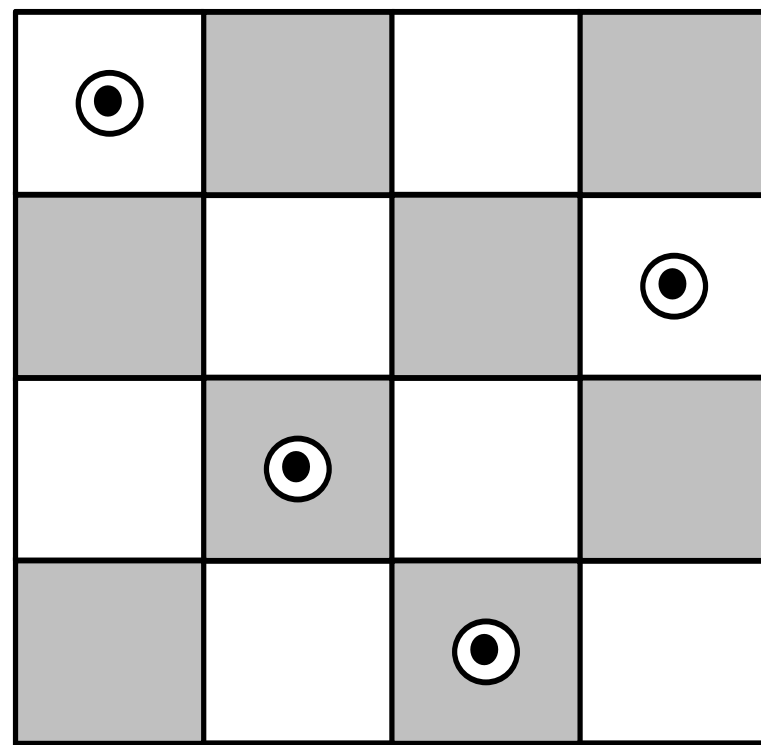
- 在 $n \times n$ 格的棋盘上放置彼此不受攻击的 n 个皇后。按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。 n 后问题等价于在 $n \times n$ 格的棋盘上放置 n 个皇后，任何2个皇后不放在同一行或同一列或同一斜线上。

1			Q					
2					Q			
3							Q	
4		Q						
5						Q		
6	Q							
7			Q					
8				Q				
	1	2	3	4	5	6	7	8

四后问题的求解过程



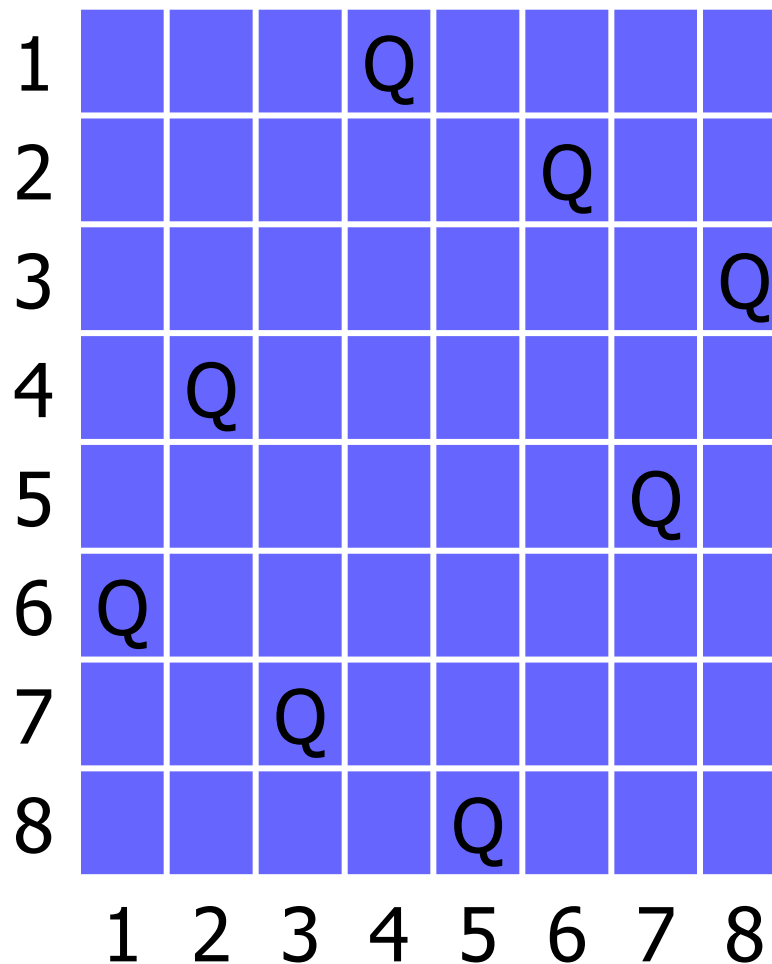
(a)



(b)

2. 问题分析

- 如何构造解空间树？



2.问题分析

- 设 x_i 表示皇后I放在棋盘的第 i 行的第 x_i 列。
- 解向量: $(x_1, x_2, \dots, x_n)$
- 显约束: $x_i=1, 2, \dots, n$
- 隐约束:
 - 1)不同列: $x_i \neq x_j$
 - 2)不处于同一正、反对角线: $|i-j| \neq |x_i - x_j|$

n后问题算法描述

```
void Queen::Backtrack(int t) //搜索解空间中第i层子树
{ if (t>n) sum++;           //sum记录当前已找到的可行方案数
  else for (int i=1;i<=n;i++) {
    x[t]=i;
    if (Place(t)) Backtrack(t+1);
  }
}
```

```
bool Queen::Place(int k)
{ for (int j=1;j<k;j++)
  if ((abs(k-j)==abs(x[j]-x[k]))||(x[j]==x[k]))
    return false;
  return true;
}
```

n后问题算法描述

```
void Queen::Backtrack(int t) //搜索解空间中第t层子树
{
    //t>n时，算法搜索至叶节点，获得一个新的n皇后互不攻击
    的放
    间中的
    先的
    树。
    bool Queen::Place(int k)
    {
        if (t > n)
        {
            for (int j=1;j<k;j++)
            {
                if ((abs(k-j)==abs(x[j]-x[k]))||(x[j]==x[k]))
                {
                    return false;
                }
            }
            if (Place(t)) Backtrack(t+1);
        }
    }
}
```

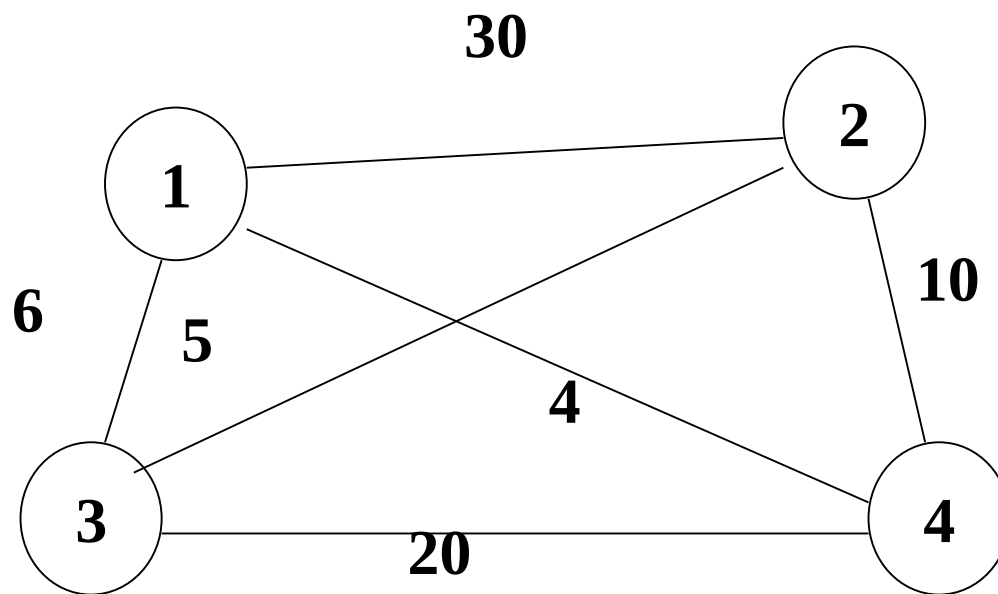

应用问题：图的着色问题

图的 m 着色问题

- 1.基本概念
- 2.问题分析
- 3.算法描述
- 4.复杂性分析

1. 基本概念

- 图的着色问题：给定一个无向连通图 G 和 m 种不同的颜色给 G 各顶点着色，每个顶点着一种颜色。问是否有使得 G 中任何一边的两个顶点着不同颜色的方法。这就是一个图的 m 可着色判定问题。
- 若一个图最少需要 m 种颜色才能使图中任何一条边连接的2个顶点着有不同颜色，则称这个数 m 为该图的色数。

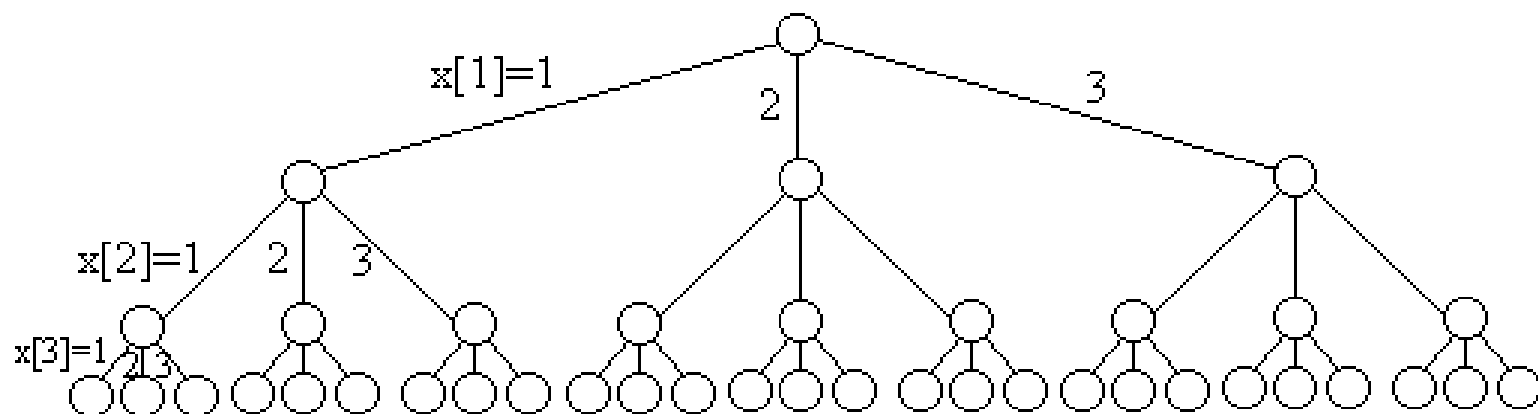


问题：如何构造解空间树？

2.问题分析

- **图的表示**: 使用邻接矩阵表示无向连通图
- **解向量**: $(x_1, x_2, \dots, x_n)$ 表示顶点 i 所着颜色 $x[i]$
- **问题的解空间**: 一棵高度为 $n+1$ 的完全 m 叉树。解空间树的第 i 层中每一结点都有 m 个儿子, 每个儿子相应于 $x[i]$ 的 m 个可能的着色之一。第 $n+1$ 层结点为叶结点。
- **可行性约束函数**: 顶点 i 与已着色的相邻顶点颜色不重复。

2.问题分析(续)



$n=3$, $m=3$ 的问题的解空间树。

图的m着色问题

```
void Color::Backtrack(int t)
{ if (t>n) {
    sum++;
    for (int i=1; i<=n; i++)
        cout << x[i] << ' ';
    cout << endl;
}
else for (int i=1;i<=m;i++) {
    x[t]=i;
    if (Ok(t))
        Backtrack(t+1);
}
}
```

图的m着色问题

```
bool Color::Ok(int k)
{ // 检查颜色可用性
  for (int j=1;j<=n;j++)
    if ((a[k][j]==1)&&(x[j]==x[k])) return false;
  return true;
}
```


复杂度分析

复杂度分析

图 m 可着色问题的解空间树中内结点

个数是 $\sum_{i=0}^{n-1} m^i$

对于每一个内结点，在最坏情况下，用ok检查当前扩展结点的每一个儿子所相应的颜色可用性需耗时 $O(mn)$ 。
因此，回溯法总的时间耗费是

$$\sum_{i=0}^{n-1} m^i (mn) = nm(m^n - 1) / (m - 1) = O(nm^n)$$

第5章 回溯法

1. 装载问题与01背包
2. 回溯算法设计步骤
3. 旅行售货员问题(TSP)
4. n 皇后问题
5. 最大团问题
6. 符号三角形
7. 回溯算法的效率

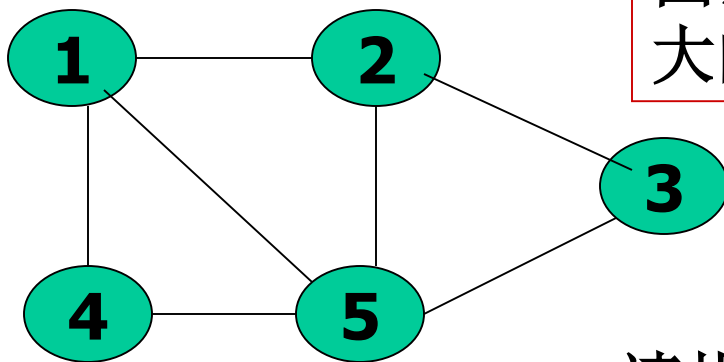
最大团问题

- 1.基本概念
- 2.问题分析
- 3.算法描述
- 4.进一步改进

1. 基本概念

- **完全子图**：给定无向图 $G=(V, E)$ 。如果 $U \subseteq V$ ，且对任意 $u, v \in U$ 有 $(u, v) \in E$ ，则称 U 是 G 的完全子图。
- **团**： G 的完全子图 U 是 G 的团当且仅当 U 不包含在 G 的更大的完全子图中。
- **G 的最大团**：是指 G 中所含顶点数最多的团。

例子



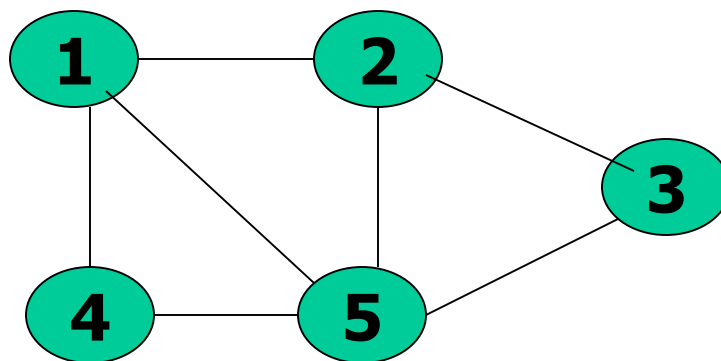
$\{1,2\}$ 是 **G** 的一个大小为**2**的完全子图，它不是团。因为它包含于 **G** 的更大的完全子图 **$\{1,2,5\}$** 之中。

请找出 **G** 的最大团？

$\{1,2,5\}$, **$\{1,4,5\}$** , **$\{2,3,5\}$** 分别是 **G** 的最大团。

1.基本概念(续)

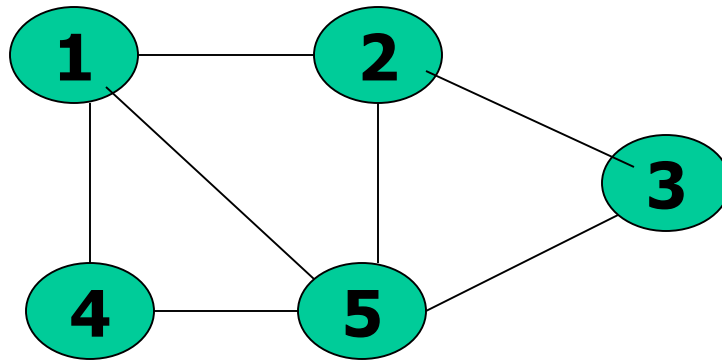
- **空子图**：如果 $U \subseteq V$ 且对任意 $u, v \in U$ 有 $(u, v) \notin E$ ，则称 U 是 G 的空子图。



{2,4}是**G**的一个空子图。

1.基本概念(续)

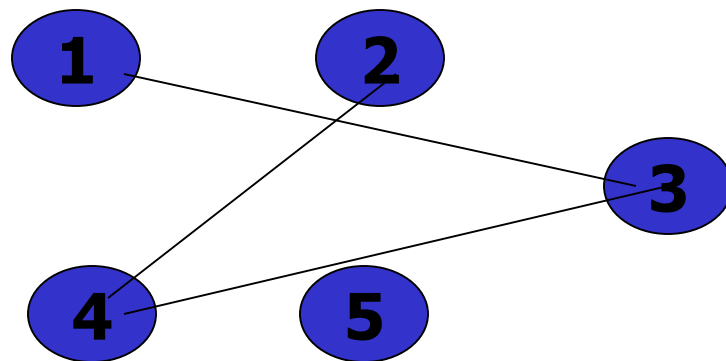
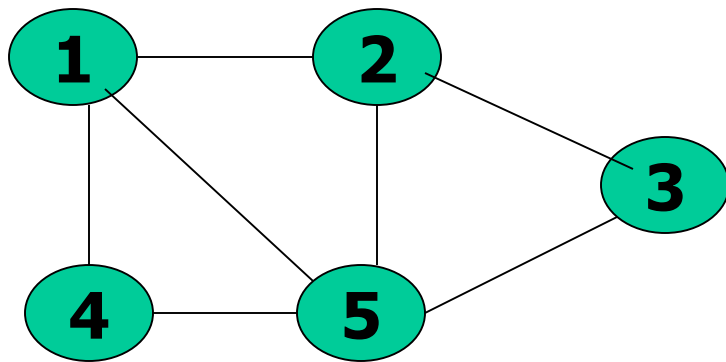
- **独立集**：G的空子图U是G的独立集当且仅当U不包含在G的更大的空子图中。
- **G的最大独立集**：是G中所含顶点数最多的独立集。



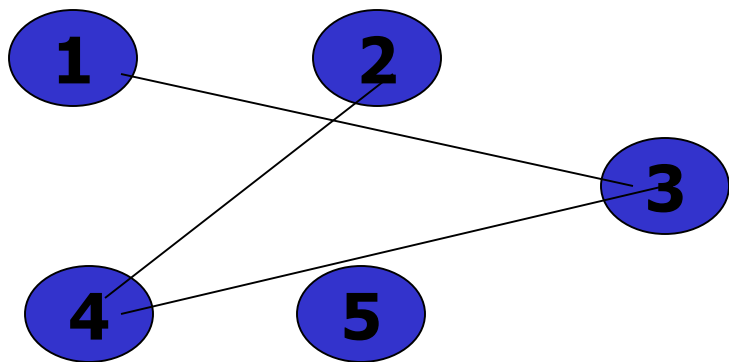
{2,4}是G的一个最大独立子集。

1.基本概念(续)

- **补图**：对于任一无向图 $G=(V, E)$ 其补图 $= (V \setminus E)$ 定义为： $V \setminus E$ 且 $(u, v) \in E \setminus E$ 当且仅当 $(u, v) \notin E$ 。



例子



{1, 2}是 $\bar{G}$ 的一个空子图，不是 G 的独立集，因为它包含在 $\bar{G}$ 的空子图**{1, 2, 5}**中。**{1, 2, 5}**是 $\bar{G}$ 的最大独立集。

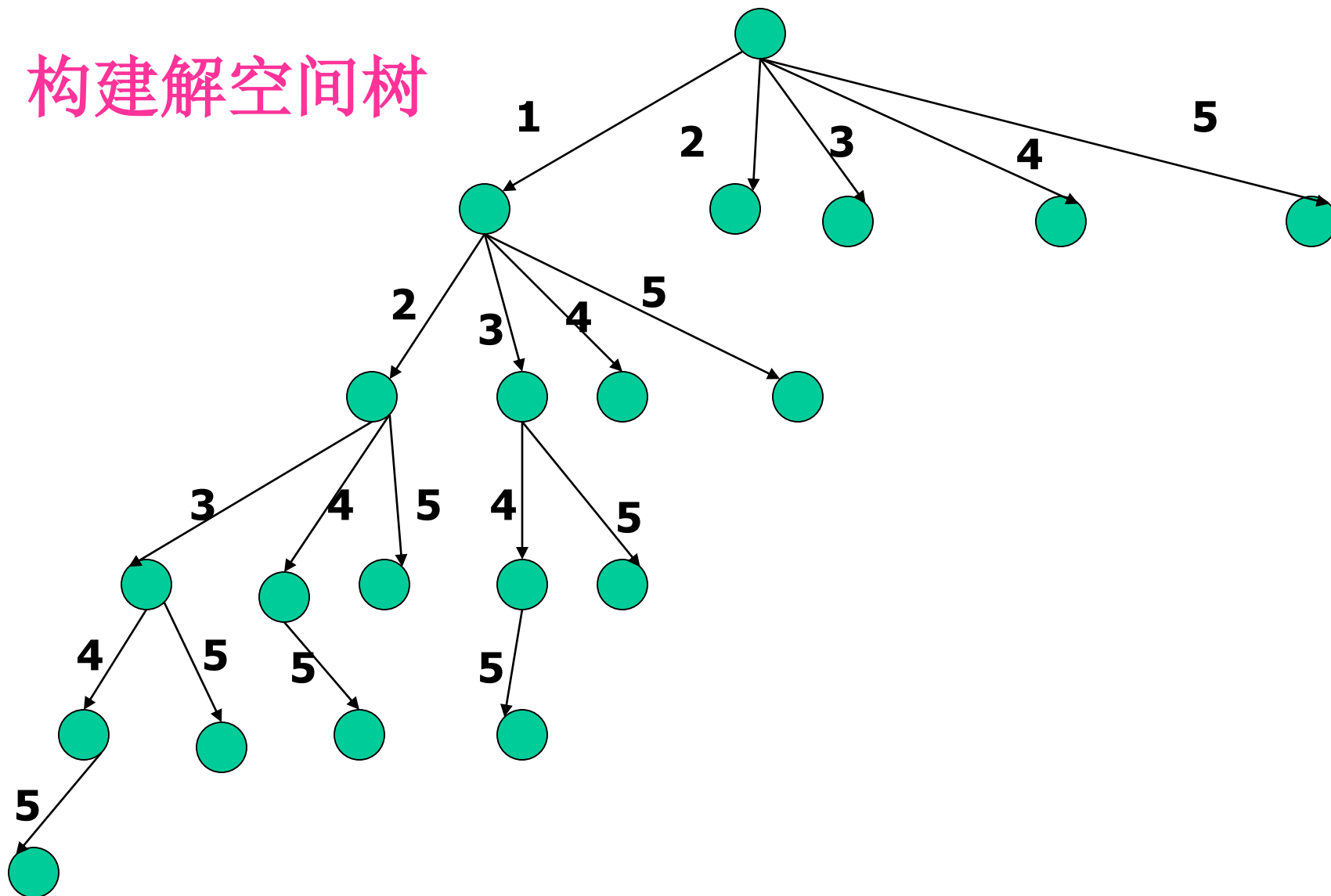
1.基本概念(续)

- U 是 G 的最大团
- 当且仅当 U 是 $\bar{G}$ 的最大独立集。

2. 问题分析

- 解空间：子集树
- 可行性约束函数：顶点 i 到已选入的顶点集中每一个顶点都有边相连。
- 上界函数：有足够多的可选择顶点使得算法有可能找到更大的团。

构建解空间树



3. 算法描述

```
void Clique::Backtrack(int i)// 计算最大团
{ if (i > n) {// 到达叶结点
    for (int j = 1; j <= n; j++) bestx[j] = x[j];
    bestn = cn; return;}
```

```
int OK = 1; // 检查顶点 i 与当前团的连接
for (int j = 1; j < i; j++)
    if (x[j] && a[i][j] == 0) {
        // i与j不相连
        OK = 0; break;}
if (OK) {// 进入左子树
    x[i] = 1; cn++;
    Backtrack(i+1);
    x[i] = 0; cn--;}
if (cn + n - i > bestn) {// 进入右子树
    x[i] = 0;
    Backtrack(i+1);}
}
```

最大团问题

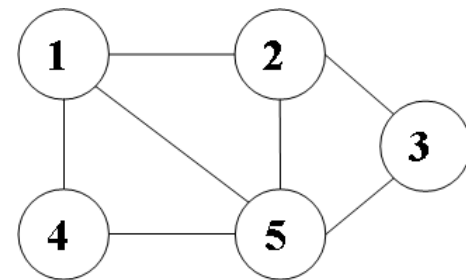
- 无向图 $G=(V,E)$. G 的完全子图称为团:

即 $U \subseteq V$ 满足 $\forall u,v \in U$, 都有 $(u,v) \in E$

- 最大团: 顶点数最多的团(子集).

- 解空间结构: 子集树

- $x[t]=1$ 或 0 表示取或不取 v_t .



bestn: 目前最大团顶点数
cn: 当前团顶点数

backtrack(t) // bestx当前最优解

1. 若 $t > n$, (若 $cn > bestn$, 更新 $bestn, bestx$), 返回

2. 若 v_t 与 $x[1:t-1]$ 中已有的 cn 个点都相连, 则

3. | $x[t]=1$, $cn++$, backtrack($t+1$), $cn--$, $x[t]=0$

4. 若 $cn+n-t > bestn$, 则

5. | $x[t]=0$, backtrack($t+1$)

复杂度分析

复杂度分析

最大团问题的回溯算法backtrack所需的计算时间显然为 $O(n2^n)$ 。

4. 进一步改进

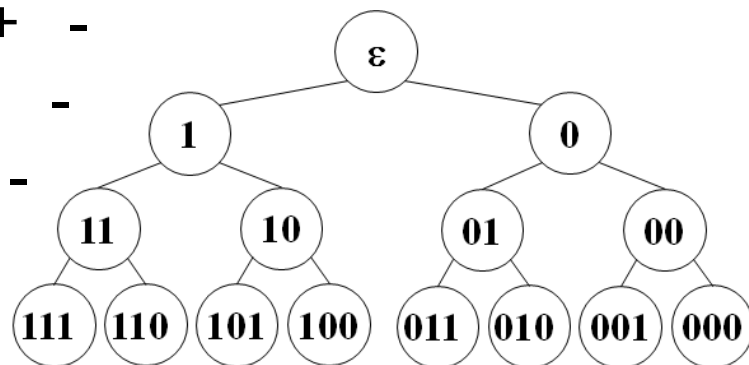
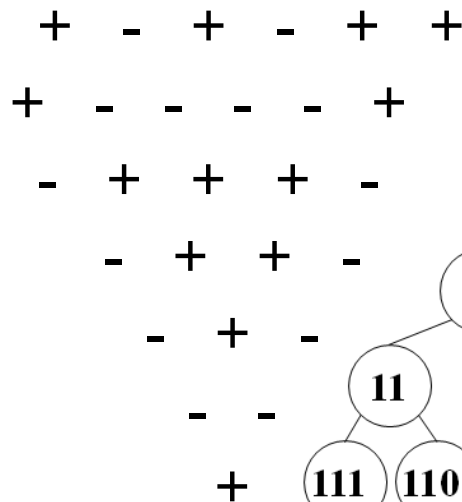
- 选择合适的搜索顺序，可以使得上界函数更有效的发挥作用。例如在搜索之前可以将顶点按度从小到大排序。这在某种意义上相当于给回溯法加入了启发性。

第5章 回溯法

1. 装载问题与01背包
2. 回溯算法设计步骤
3. 旅行售货员问题(TSP)
4. n 皇后问题
5. 最大团问题
6. 符号三角形
7. 回溯算法的效率

符号三角形

- 由符号 “+”, “-” 组成
- 第一行有 n 个符号
- 两个同号下面是 “+”
- 两个异号下面是 “-”
- 右图有 14 “+”, 14 “-”
- 问题: 给定 n , 求 “+” “-” 个数相同的符号三角形个数
- 如何遍历所有符号三角形? 约束条件? 限界条件?
- 第一行取遍所有长为 n 的 +- 符号串: 子集树
- 限界条件: 0, 1 的个数都不能超过 $n(n+1)/4$



符号三角形

- **sum** 累计+-个数相同的符号三角形个数
- 若 $n(n+1)/2$ 是奇数 则 $\text{sum}=0$, 返回.
- $\text{half} = n(n+1)/4$
- **p** 维护当前符号三角形
- $p[1][t] = x[t]$
- 对 $j=2:t$, 依次计算 $p[j][t-j+1]$

```
+  +  -  +  -  +  +
  +  -  -  -  -  +
    -  +  +  +  -
      -  +  +  -
        -  +  -
          -  -
            +
```

符号三角形

//sum记+-个数相同的符号三角形个数

//half= $n(n+1)/4$, p维护当前符号三角形

backtrack (t)

1. 若 $\text{count} > \text{half}$ 或 $t(t-1)/2 - \text{count} > \text{half}$, 返回

2. 若 $t > n$, $\text{sum}++$, 返回

3. 对 $i = 0 : 1$,

4. | $p[1][t] = i$; $\text{count} += i$;

5. | 对 $j = 2 : t$,

6. | | $p[j][t-j+1] = p[j-1][t-j+1] \wedge p[j-1][t-j+2]$;

7. | | $\text{count} += p[j][t-j+1]$;

8. | backtrack($t+1$);

9. | 对 $j = 2 : t$, $\text{count} -= p[j][t-j+1]$;

10. | $\text{count} -= i$; //可与9合并

0 0 0

0 0

0

0 0 1

0 1

1

0 1 0

1 1

0

0 1 1

1 0

1

符号三角形

//sum记+-个数相同的符号三角形个数

//half= $n(n+1)/4$, p维护当前符号三角形

backtrack (t)

1. 若 $t > n$, sum++, 返回

2. 对 $i = 0 : 1$,

3. | $p[1][t] = i$; count+=i;

4. | 对 $j = 2 : t$,

5. | | $p[j][t-j+1] = p[j-1][t-j+1] \wedge p[j-1][t-j+2]$;

6. | | count+=p[j][t-j+1];

7. | 若 $\text{count} \leq \text{half}$ 且 $(t+1)t/2 - \text{count} \leq \text{half}$

8. | | backtrack(t+1);

9. | 对 $j = 1 : t$, count-=p[j][t-j+1];

0 0 0

0 0

0

0 0 1

0 1

1

0 1 0

1 1

0

0 1 1

1 0

1

第5章 回溯法

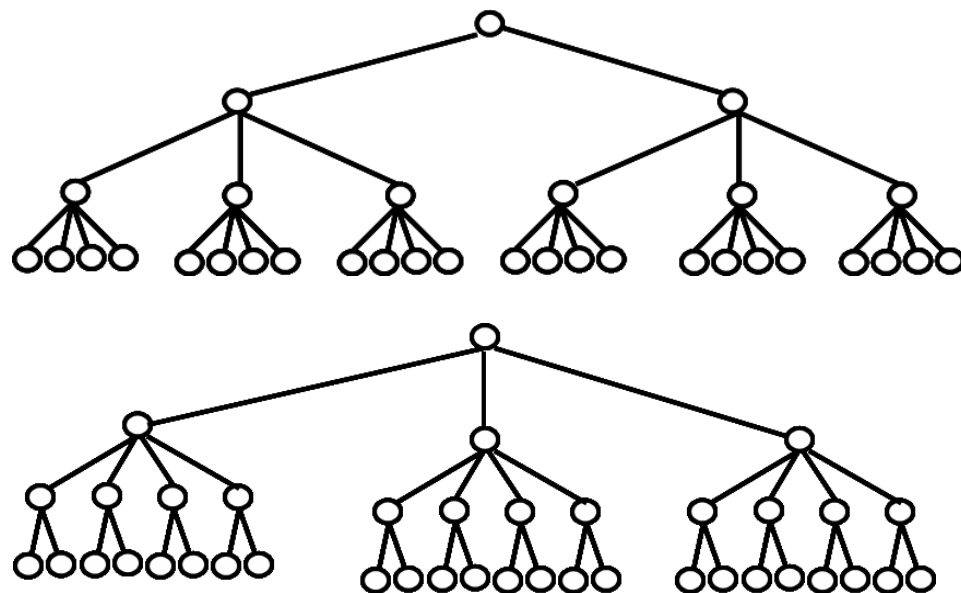
1. 装载问题与01背包
2. 回溯算法设计步骤
3. 旅行售货员问题(TSP)
4. n 皇后问题
5. 最大团问题
6. 符号三角形
7. 回溯算法的效率

回溯法的效率分析

影响回溯算法效率的因素

1. 每个顶点的产生时间
2. 计算剪枝函数的时间
3. 剪枝后剩余顶点个数

剪枝函数的设计与平衡? 更好的剪枝会增加计算时间



回溯法的效率举例

教材[王]中对n皇后问题的回溯效率进行了概率估计
对于n=8的情形

估计搜索节点数/总节点数 $\approx 1.55\%$

总节点数 = $\sum_{i=1}^n n!/(n-i)! = 109601$ (n=8)

搜索节点数估计:

记第i层x[i]的可选列数为 m_i ,

随机选一可选列进入下一层

得节点数 $1 + m_1 + m_1 m_2 + m_1 m_2 m_3 + \dots$

多次取平均得1702

$m_1=8$; $m_2=5$: 因为2可选4-8列; $m_3=4$: 因为3可选1,6,7,8;
 $m_4=3$: 因为4可选3,7,8; $m_5=2$: 因为5可选5,8;

		1					
					2		
	3						
						4	
5							
			6				
							7
				8			

2329

回溯法的效率举例

		1					
	X	X	X		2		
X	3	X		X	X	X	
X	X	X	X		X	4	X
5	X	X	X		X	X	X
X	X	X	6	X	X	X	X
X	X	X	X	X	X	X	7
X	X	X	X	8	X	X	X

$$1+8+8*5+8*5*3$$

$$+8*5*3*2$$

$$+8*5*3*2*2$$

$$+8*5*3*2*2*1$$

$$+8*5*3*2*2*1*1$$

$$+8*5*3*2*2*1*1*1$$

$$=2329$$